

14.6 Shortest Paths

As we saw in Section 14.3.3, the breadth-first search strategy can be used to find a shortest path from some starting vertex to every other vertex in a connected graph. This approach makes sense in cases where each edge is as good as any other, but there are many situations where this approach is not appropriate.

For example, we might want to use a graph to represent the roads between cities, and we might be interested in finding the fastest way to travel cross-country. In this case, it is probably not appropriate for all the edges to be equal to each other, for some inter-city distances will likely be much larger than others. Likewise, we might be using a graph to represent a computer network (such as the Internet), and we might be interested in finding the fastest way to route a data packet between two computers. In this case, it again may not be appropriate for all the edges to be equal to each other, for some connections in a computer network are typically much faster than others (for example, some edges might represent low-bandwidth connections, while others might represent high-speed, fiber-optic connections). It is natural, therefore, to consider graphs whose edges are not weighted equally.

14.6.1 Weighted Graphs

A **weighted graph** is a graph that has a numeric (for example, integer) label $w(e)$ associated with each edge e , called the **weight** of edge e . For $e = (u, v)$, we let notation $w(u, v) = w(e)$. We show an example of a weighted graph in Figure 14.14.

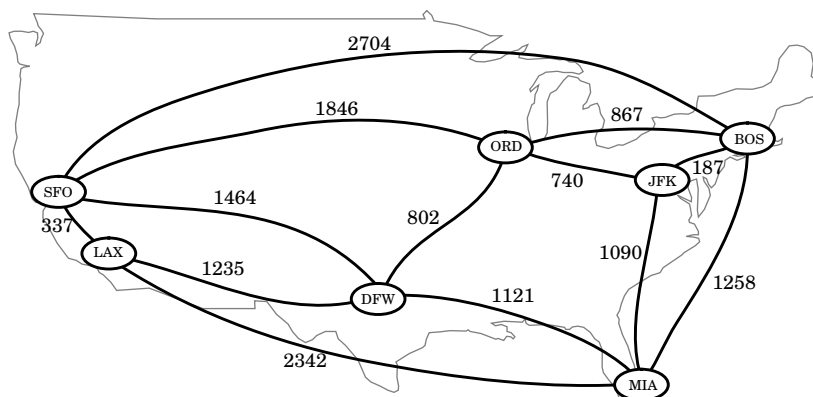


Figure 14.14: A weighted graph whose vertices represent major U.S. airports and whose edge weights represent distances in miles. This graph has a path from JFK to LAX of total weight 2,777 (going through ORD and DFW). This is the minimum-weight path in the graph from JFK to LAX.

Defining Shortest Paths in a Weighted Graph

Let G be a weighted graph. The **length** (or weight) of a path is the sum of the weights of the edges of P . That is, if $P = ((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k))$, then the length of P , denoted $w(P)$, is defined as

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$

The **distance** from a vertex u to a vertex v in G , denoted $d(u, v)$, is the length of a minimum-length path (also called **shortest path**) from u to v , if such a path exists.

People often use the convention that $d(u, v) = \infty$ if there is no path at all from u to v in G . Even if there is a path from u to v in G , however, if there is a cycle in G whose total weight is negative, the distance from u to v may not be defined. For example, suppose vertices in G represent cities, and the weights of edges in G represent how much money it costs to go from one city to another. If someone were willing to actually pay us to go from say JFK to ORD, then the “cost” of the edge (JFK, ORD) would be negative. If someone else were willing to pay us to go from ORD to JFK, then there would be a negative-weight cycle in G and distances would no longer be defined. That is, anyone could now build a path (with cycles) in G from any city A to another city B that first goes to JFK and then cycles as many times as he or she likes from JFK to ORD and back, before going on to B . The existence of such paths would allow us to build arbitrarily low negative-cost paths (and, in this case, make a fortune in the process). But distances cannot be arbitrarily low negative numbers. Thus, any time we use edge weights to represent distances, we must be careful not to introduce any negative-weight cycles.

Suppose we are given a weighted graph G , and we are asked to find a shortest path from some vertex s to each other vertex in G , viewing the weights on the edges as distances. In this section, we explore efficient ways of finding all such shortest paths, if they exist. The first algorithm we discuss is for the simple, yet common, case when all the edge weights in G are nonnegative (that is, $w(e) \geq 0$ for each edge e of G); hence, we know in advance that there are no negative-weight cycles in G . Recall that the special case of computing a shortest path when all weights are equal to one was solved with the BFS traversal algorithm presented in Section 14.3.3.

There is an interesting approach for solving this **single-source** problem based on the **greedy method** design pattern (Section 13.4.2). Recall that in this pattern we solve the problem at hand by repeatedly selecting the best choice from among those available in each iteration. This paradigm can often be used in situations where we are trying to optimize some cost function over a collection of objects. We can add objects to our collection, one at a time, always picking the next one that optimizes the function from among those yet to be chosen.

14.6.2 Dijkstra's Algorithm

The main idea in applying the greedy method pattern to the single-source shortest-path problem is to perform a “weighted” breadth-first search starting at the source vertex s . In particular, we can use the greedy method to develop an algorithm that iteratively grows a “cloud” of vertices out of s , with the vertices entering the cloud in order of their distances from s . Thus, in each iteration, the next vertex chosen is the vertex outside the cloud that is closest to s . The algorithm terminates when no more vertices are outside the cloud (or when those outside the cloud are not connected to those within the cloud), at which point we have a shortest path from s to every vertex of G that is reachable from s . This approach is a simple, but nevertheless powerful, example of the greedy method design pattern. Applying the greedy method to the single-source, shortest-path problem, results in an algorithm known as *Dijkstra's algorithm*.

Edge Relaxation

Let us define a label $D[v]$ for each vertex v in V , which we use to approximate the distance in G from s to v . The meaning of these labels is that $D[v]$ will always store the length of the best path we have found *so far* from s to v . Initially, $D[s] = 0$ and $D[v] = \infty$ for each $v \neq s$, and we define the set C , which is our “**cloud**” of vertices, to initially be the empty set. At each iteration of the algorithm, we select a vertex u not in C with smallest $D[u]$ label, and we pull u into C . (In general, we will use a priority queue to select among the vertices outside the cloud.) In the very first iteration we will, of course, pull s into C . Once a new vertex u is pulled into C , we then update the label $D[v]$ of each vertex v that is adjacent to u and is outside of C , to reflect the fact that there may be a new and better way to get to v via u . This update operation is known as a *relaxation* procedure, for it takes an old estimate and checks if it can be improved to get closer to its true value. The specific edge relaxation operation is as follows:

Edge Relaxation:

```

if  $D[u] + w(u, v) < D[v]$  then
     $D[v] = D[u] + w(u, v)$ 

```

Algorithm Description and Example

We give the pseudo-code for Dijkstra's algorithm in Code Fragment 14.12, and illustrate several iterations of Dijkstra's algorithm in Figures 14.15 through 14.17.

Algorithm ShortestPath(G, s):

Input: A weighted graph G with nonnegative edge weights, and a distinguished vertex s of G .

Output: The length of a shortest path from s to v for each vertex v of G .

Initialize $D[s] = 0$ and $D[v] = \infty$ for each vertex $v \neq s$.

Let a priority queue Q contain all the vertices of G using the D labels as keys.

while Q is not empty **do**

 {pull a new vertex u into the cloud}

$u =$ value returned by $Q.remove_min()$

for each vertex v adjacent to u such that v is in Q **do**

 {perform the *relaxation* procedure on edge (u, v) }

if $D[u] + w(u, v) < D[v]$ **then**

$D[v] = D[u] + w(u, v)$

 Change to $D[v]$ the key of vertex v in Q .

return the label $D[v]$ of each vertex v

Code Fragment 14.12: Pseudo-code for Dijkstra's algorithm, solving the single-source shortest-path problem.

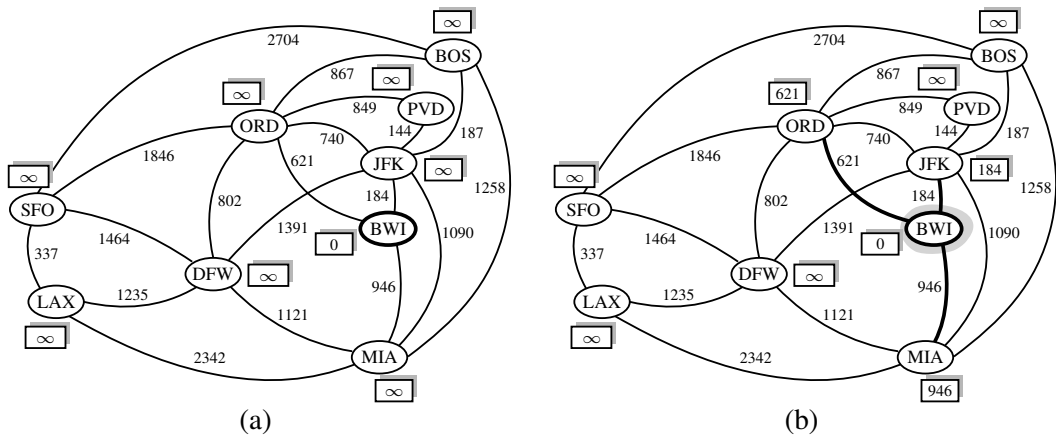


Figure 14.15: An execution of Dijkstra's algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

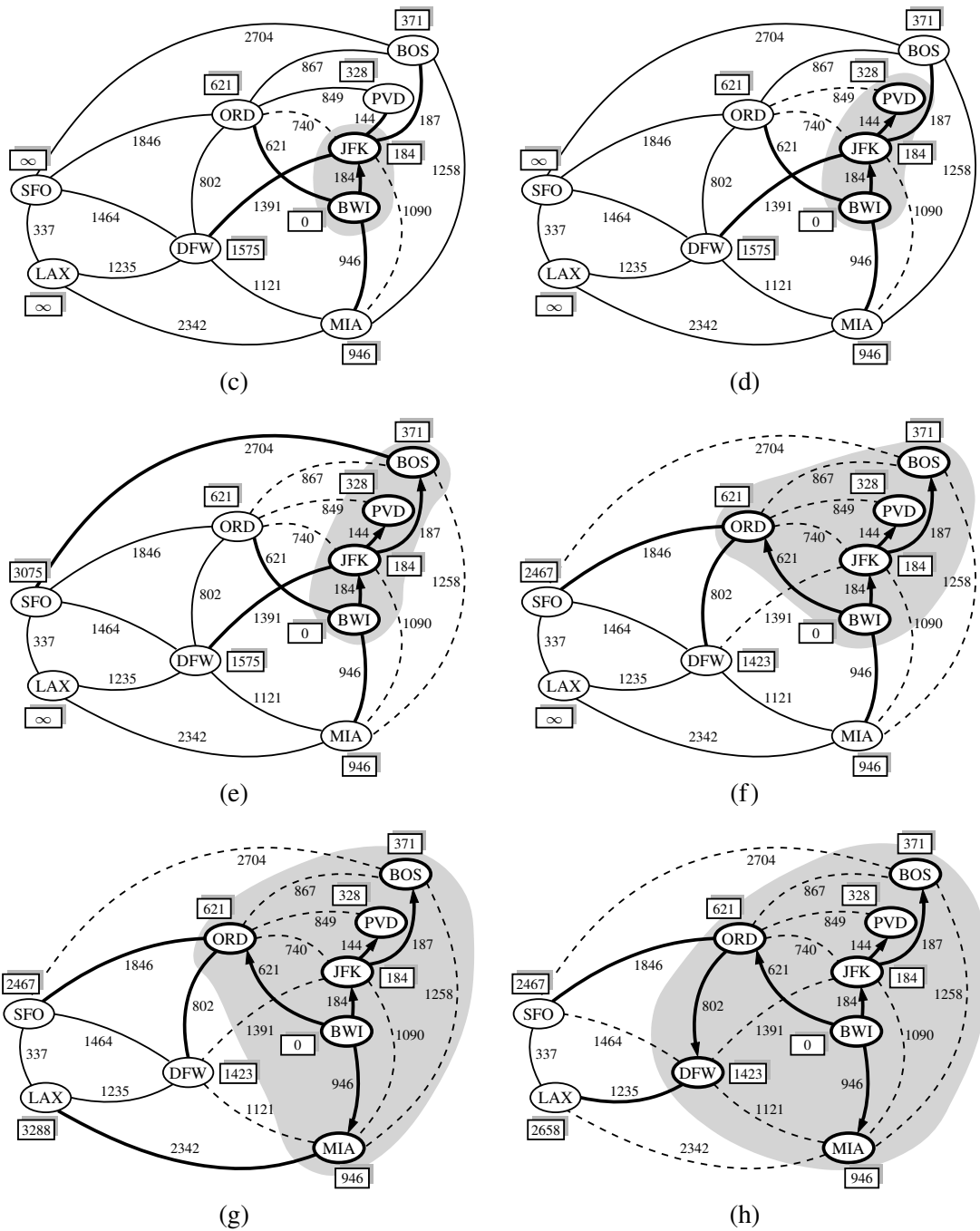


Figure 14.16: An example execution of Dijkstra's algorithm. (Continued from Figure 14.15; continued in Figure 14.17.)

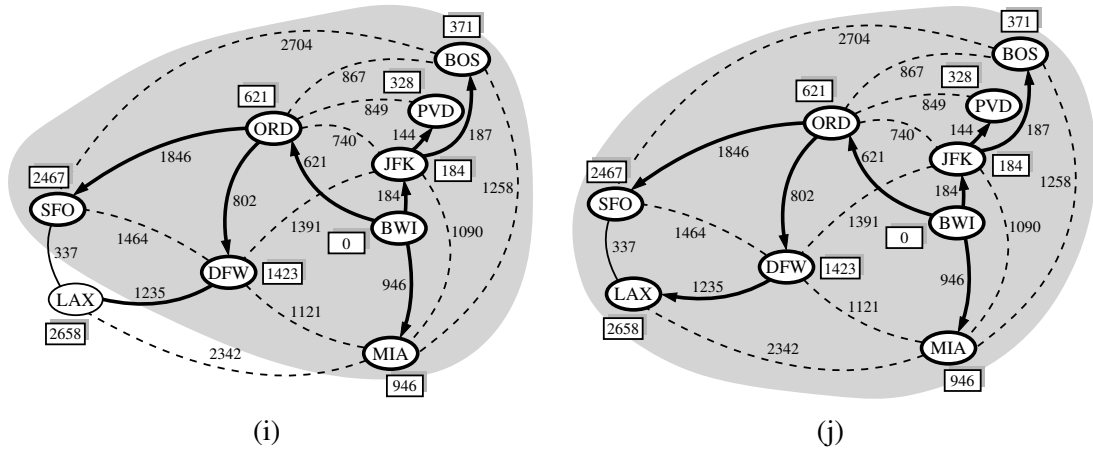


Figure 14.17: An example execution of Dijkstra's algorithm. (Continued from Figure 14.16.)

Why It Works

The interesting aspect of the Dijkstra algorithm is that, at the moment a vertex u is pulled into C , its label $D[u]$ stores the correct length of a shortest path from v to u . Thus, when the algorithm terminates, it will have computed the shortest-path distance from s to every vertex of G . That is, it will have solved the single-source shortest-path problem.

It is probably not immediately clear why Dijkstra's algorithm correctly finds the shortest path from the start vertex s to each other vertex u in the graph. Why is it that the distance from s to u is equal to the value of the label $D[u]$ at the time vertex u is removed from the priority queue Q and added to the cloud C ? The answer to this question depends on there being no negative-weight edges in the graph, for it allows the greedy method to work correctly, as we show in the proposition that follows.

Proposition 14.23: *In Dijkstra's algorithm, whenever a vertex v is pulled into the cloud, the label $D[v]$ is equal to $d(s, v)$, the length of a shortest path from s to v .*

Justification: Suppose that $D[v] > d(s, v)$ for some vertex v in V , and let z be the *first* vertex the algorithm pulled into the cloud C (that is, removed from Q) such that $D[z] > d(s, z)$. There is a shortest path P from s to z (for otherwise $d(s, z) = \infty = D[z]$). Let us therefore consider the moment when z is pulled into C , and let y be the first vertex of P (when going from s to z) that is not in C at this moment. Let x be the predecessor of y in path P (note that we could have $x = s$). (See Figure 14.18.) We know, by our choice of y , that x is already in C at this point.

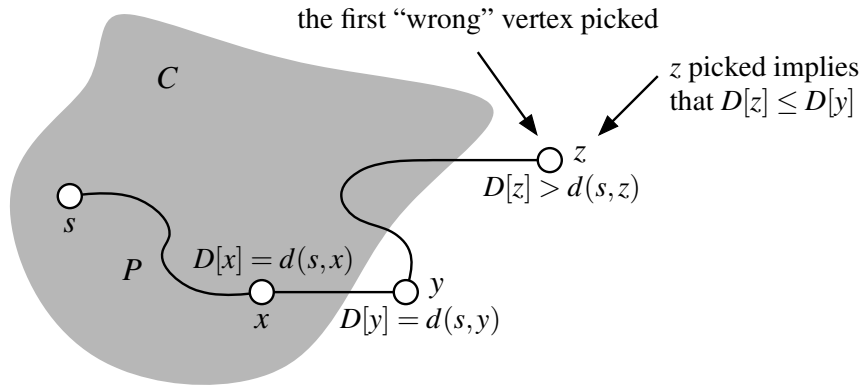


Figure 14.18: A schematic illustration for the justification of Proposition 14.23.

Moreover, $D[x] = d(s, x)$, since z is the *first* incorrect vertex. When x was pulled into C , we tested (and possibly updated) $D[y]$ so that we had at that point

$$D[y] \leq D[x] + w(x, y) = d(s, x) + w(x, y).$$

But since y is the next vertex on the shortest path from s to z , this implies that

$$D[y] = d(s, y).$$

But we are now at the moment when we are picking z , not y , to join C ; hence,

$$D[z] \leq D[y].$$

It should be clear that a subpath of a shortest path is itself a shortest path. Hence, since y is on the shortest path from s to z ,

$$d(s, y) + d(y, z) = d(s, z).$$

Moreover, $d(y, z) \geq 0$ because there are no negative-weight edges. Therefore,

$$D[z] \leq D[y] = d(s, y) \leq d(s, y) + d(y, z) = d(s, z).$$

But this contradicts the definition of z ; hence, there can be no such vertex z . ■

The Running Time of Dijkstra's Algorithm

In this section, we analyze the time complexity of Dijkstra's algorithm. We denote with n and m the number of vertices and edges of the input graph G , respectively. We assume that the edge weights can be added and compared in constant time. Because of the high level of the description we gave for Dijkstra's algorithm in Code Fragment 14.12, analyzing its running time requires that we give more details on its implementation. Specifically, we should indicate the data structures used and how they are implemented.

Let us first assume that we are representing the graph G using an adjacency list or adjacency map structure. This data structure allows us to step through the vertices adjacent to u during the relaxation step in time proportional to their number. Therefore, the time spent in the management of the nested **for** loop, and the number of iterations of that loop, is

$$\sum_{u \text{ in } V_G} \text{outdeg}(u),$$

which is $O(m)$ by Proposition 14.9. The outer **while** loop executes $O(n)$ times, since a new vertex is added to the cloud during each iteration. This still does not settle all the details for the algorithm analysis, however, for we must say more about how to implement the other principal data structure in the algorithm—the priority queue Q .

Referring back to Code Fragment 14.12 in search of priority queue operations, we find that n vertices are originally inserted into the priority queue; since these are the only insertions, the maximum size of the queue is n . In each of n iterations of the **while** loop, a call to `remove_min` is made to extract the vertex u with smallest D label from Q . Then, for each neighbor v of u , we perform an edge relaxation, and may potentially update the key of v in the queue. Thus, we actually need an implementation of an *adaptable priority queue* (Section 9.5), in which case the key of a vertex v is changed using the method `update( $\ell, k$ )`, where ℓ is the locator for the priority queue entry associated with vertex v . In the worst case, there could be one such update for each edge of the graph. Overall, the running time of Dijkstra's algorithm is bounded by the sum of the following:

- n insertions into Q .
- n calls to the `remove_min` method on Q .
- m calls to the `update` method on Q .

If Q is an adaptable priority queue implemented as a heap, then each of the above operations run in $O(\log n)$, and so the overall running time for Dijkstra's algorithm is $O((n + m) \log n)$. Note that if we wish to express the running time as a function of n only, then it is $O(n^2 \log n)$ in the worst case.

Let us now consider an alternative implementation for the adaptable priority queue Q using an unsorted sequence. (See Exercise P-9.58.) This, of course, requires that we spend $O(n)$ time to extract the minimum element, but it affords very fast key updates, provided Q supports location-aware entries (Section 9.5.1). Specifically, we can implement each key update done in a relaxation step in $O(1)$ time—we simply change the key value once we locate the entry in Q to update. Hence, this implementation results in a running time that is $O(n^2 + m)$, which can be simplified to $O(n^2)$ since G is simple.

Comparing the Two Implementations

We have two choices for implementing the adaptable priority queue with location-aware entries in Dijkstra's algorithm: a heap implementation, which yields a running time of $O((n + m) \log n)$, and an unsorted sequence implementation, which yields a running time of $O(n^2)$. Since both implementations would be fairly simple to code, they are about equal in terms of the programming sophistication needed. These two implementations are also about equal in terms of the constant factors in their worst-case running times. Looking only at these worst-case times, we prefer the heap implementation when the number of edges in the graph is small (that is, when $m < n^2 / \log n$), and we prefer the sequence implementation when the number of edges is large (that is, when $m > n^2 / \log n$).

Proposition 14.24: *Given a weighted graph G with n vertices and m edges, such that the weight of each edge is nonnegative, and a vertex s of G , Dijkstra's algorithm can compute the distance from s to all other vertices of G in the better of $O(n^2)$ or $O((n + m) \log n)$ time.*

We note that an advanced priority queue implementation, known as a *Fibonacci heap*, can be used to implement Dijkstra's algorithm in $O(m + n \log n)$ time.

Programming Dijkstra's Algorithm in Python

Having given a pseudo-code description of Dijkstra's algorithm, let us now present Python code for performing Dijkstra's algorithm, assuming we are given a graph whose edge elements are nonnegative integer weights. Our implementation of the algorithm is in the form of a function, `shortest_path_lengths`, that takes a graph and a designated source vertex as parameters. (See Code Fragment 14.13.) It returns a dictionary, named `cloud`, mapping each vertex v that is reachable from the source to its shortest-path distance $d(s, v)$. We rely on our `AdaptableHeapPriorityQueue` developed in Section 9.5.2 as an adaptable priority queue.

As we have done with other algorithms in this chapter, we rely on dictionaries to map vertices to associated data (in this case, mapping v to its distance bound $D[v]$ and its adaptable priority queue locator). The expected $O(1)$ -time access to elements of these dictionaries could be converted to worst-case bounds, either by numbering vertices from 0 to $n - 1$ to use as indices into a list, or by storing the information within each vertex's element.

The pseudo-code for Dijkstra's algorithm begins by assigning $d[v] = \infty$ for each v other than the source. We rely on the special value `float('inf')` in Python to provide a numeric value that represents positive infinity. However, we avoid including vertices with this "infinite" distance in the resulting `cloud` that is returned by the function. The use of this numeric limit could be avoided altogether by waiting to add a vertex to the priority queue until after an edge that reaches it is relaxed. (See Exercise C-14.64.)

```

1  def shortest_path_lengths(g, src):
2      """ Compute shortest-path distances from src to reachable vertices of g.
3
4      Graph g can be undirected or directed, but must be weighted such that
5      e.element() returns a numeric weight for each edge e.
6
7      Return dictionary mapping each reachable vertex to its distance from src.
8      """
9      d = { }                                # d[v] is upper bound from s to v
10     cloud = { }                             # map reachable v to its d[v] value
11     pq = AdaptableHeapPriorityQueue( )      # vertex v will have key d[v]
12     pqlocator = { }                         # map from vertex to its pq locator
13
14     # for each vertex v of the graph, add an entry to the priority queue, with
15     # the source having distance 0 and all others having infinite distance
16     for v in g.vertices():
17         if v is src:
18             d[v] = 0
19         else:
20             d[v] = float('inf')             # syntax for positive infinity
21             pqlocator[v] = pq.add(d[v], v)   # save locator for future updates
22
23     while not pq.is_empty():
24         key, u = pq.remove_min()
25         cloud[u] = key                       # its correct d[u] value
26         del pqlocator[u]                    # u is no longer in pq
27         for e in g.incident_edges(u):      # outgoing edges (u,v)
28             v = e.opposite(u)
29             if v not in cloud:
30                 # perform relaxation step on edge (u,v)
31                 wgt = e.element()
32                 if d[u] + wgt < d[v]:        # better path to v?
33                     d[v] = d[u] + wgt      # update the distance
34                     pq.update(pqlocator[v], d[v], v) # update the pq entry
35
36     return cloud                            # only includes reachable vertices

```

Code Fragment 14.13: Python implementation of Dijkstra’s algorithm for computing the shortest-path distances from a single source. We assume that `e.element()` for edge `e` represents the weight of that edge.

Reconstructing the Shortest-Path Tree

Our pseudo-code description of Dijkstra's algorithm in Code Fragment 14.12, and our implementation in Code Fragment 14.13, computes the value $d[v]$, for each vertex v , that is the length of the shortest path from the source vertex s to v . However, those forms of the algorithm do not explicitly compute the actual paths that achieve those distances. The collection of all shortest paths emanating from source s can be compactly represented by what is known as the *shortest-path tree*. The paths form a rooted tree because if a shortest path from s to v passes through an intermediate vertex u , it must begin with a shortest path from s to u .

In this section, we demonstrate that the shortest-path tree rooted at source s can be reconstructed in $O(n + m)$ time, given the set of $d[v]$ values produced by Dijkstra's algorithm using s as the source. As we did when representing the DFS and BFS trees, we will map each vertex $v \neq s$ to a parent u (possibly, $u = s$), such that u is the vertex immediately before v on a shortest path from s to v . If u is the vertex just before v on the shortest path from s to v , it must be that

$$d[u] + w(u, v) = d[v].$$

Conversely, if the above equation is satisfied, then the shortest path from s to u , followed by the edge (u, v) is a shortest path to v .

Our implementation in Code Fragment 14.14 reconstructs the tree based on this logic, testing all *incoming* edges to each vertex v , looking for a (u, v) that satisfies the key equation. The running time is $O(n + m)$, as we consider each vertex and all incoming edges to those vertices. (See Proposition 14.9.)

```

1  def shortest_path_tree(g, s, d):
2      """Reconstruct shortest-path tree rooted at vertex s, given distance map d.
3
4      Return tree as a map from each reachable vertex v (other than s) to the
5      edge e=(u,v) that is used to reach v from its parent u in the tree.
6      """
7      tree = { }
8      for v in d:
9          if v is not s:
10             for e in g.incident_edges(v, False):          # consider INCOMING edges
11                 u = e.opposite(v)
12                 wgt = e.element()
13                 if d[v] == d[u] + wgt:
14                     tree[v] = e                             # edge e is used to reach v
15      return tree

```

Code Fragment 14.14: Python function that reconstructs the shortest paths, based on knowledge of the single-source distances.

14.7 Minimum Spanning Trees

Suppose we wish to connect all the computers in a new office building using the least amount of cable. We can model this problem using an undirected, weighted graph G whose vertices represent the computers, and whose edges represent all the possible pairs (u, v) of computers, where the weight $w(u, v)$ of edge (u, v) is equal to the amount of cable needed to connect computer u to computer v . Rather than computing a shortest-path tree from some particular vertex v , we are interested instead in finding a tree T that contains all the vertices of G and has the minimum total weight over all such trees. Algorithms for finding such a tree are the focus of this section.

Problem Definition

Given an undirected, weighted graph G , we are interested in finding a tree T that contains all the vertices in G and minimizes the sum

$$w(T) = \sum_{(u,v) \text{ in } T} w(u,v).$$

A tree, such as this, that contains every vertex of a connected graph G is said to be a **spanning tree**, and the problem of computing a spanning tree T with smallest total weight is known as the **minimum spanning tree** (or **MST**) problem.

The development of efficient algorithms for the minimum spanning tree problem predates the modern notion of computer science itself. In this section, we discuss two classic algorithms for solving the MST problem. These algorithms are both applications of the **greedy method**, which, as was discussed briefly in the previous section, is based on choosing objects to join a growing collection by iteratively picking an object that minimizes some cost function. The first algorithm we discuss is the Prim-Jarník algorithm, which grows the MST from a single root vertex, much in the same way as Dijkstra's shortest-path algorithm. The second algorithm we discuss is Kruskal's algorithm, which "grows" the MST in clusters by considering edges in nondecreasing order of their weights.

In order to simplify the description of the algorithms, we assume, in the following, that the input graph G is undirected (that is, all its edges are undirected) and simple (that is, it has no self-loops and no parallel edges). Hence, we denote the edges of G as unordered vertex pairs (u, v) .

Before we discuss the details of these algorithms, however, let us give a crucial fact about minimum spanning trees that forms the basis of the algorithms.

A Crucial Fact about Minimum Spanning Trees

The two MST algorithms we discuss are based on the greedy method, which in this case depends crucially on the following fact. (See Figure 14.19.)

e Belongs to a Minimum Spanning Tree

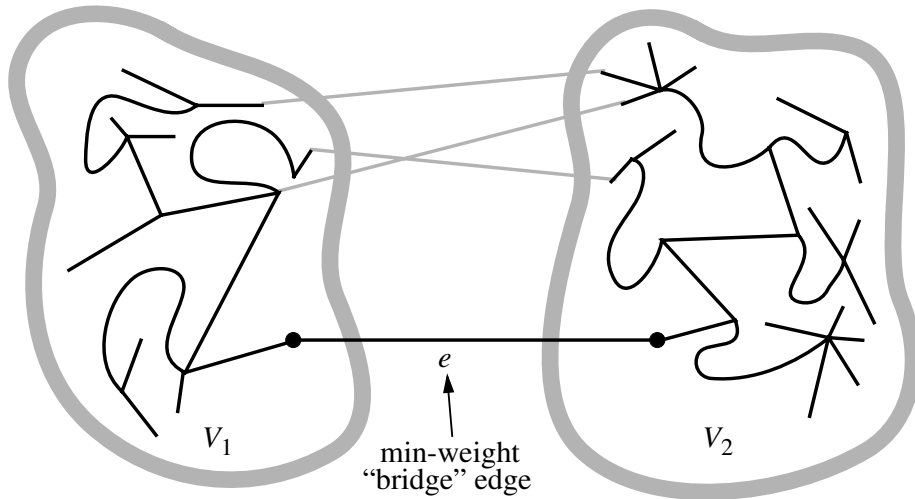


Figure 14.19: An illustration of the crucial fact about minimum spanning trees.

Proposition 14.25: Let G be a weighted connected graph, and let V_1 and V_2 be a partition of the vertices of G into two disjoint nonempty sets. Furthermore, let e be an edge in G with minimum weight from among those with one endpoint in V_1 and the other in V_2 . There is a minimum spanning tree T that has e as one of its edges.

Justification: Let T be a minimum spanning tree of G . If T does not contain edge e , the addition of e to T must create a cycle. Therefore, there is some edge $f \neq e$ of this cycle that has one endpoint in V_1 and the other in V_2 . Moreover, by the choice of e , $w(e) \leq w(f)$. If we remove f from $T \cup \{e\}$, we obtain a spanning tree whose total weight is no more than before. Since T was a minimum spanning tree, this new tree must also be a minimum spanning tree. ■

In fact, if the weights in G are distinct, then the minimum spanning tree is unique; we leave the justification of this less crucial fact as an exercise (C-14.65). In addition, note that Proposition 14.25 remains valid even if the graph G contains negative-weight edges or negative-weight cycles, unlike the algorithms we presented for shortest paths.

14.7.1 Prim-Jarník Algorithm

In the Prim-Jarník algorithm, we grow a minimum spanning tree from a single cluster starting from some “root” vertex s . The main idea is similar to that of Dijkstra’s algorithm. We begin with some vertex s , defining the initial “cloud” of vertices C . Then, in each iteration, we choose a minimum-weight edge $e = (u, v)$, connecting a vertex u in the cloud C to a vertex v outside of C . The vertex v is then brought into the cloud C and the process is repeated until a spanning tree is formed. Again, the crucial fact about minimum spanning trees comes into play, for by always choosing the smallest-weight edge joining a vertex inside C to one outside C , we are assured of always adding a valid edge to the MST.

To efficiently implement this approach, we can take another cue from Dijkstra’s algorithm. We maintain a label $D[v]$ for each vertex v outside the cloud C , so that $D[v]$ stores the weight of the minimum observed edge for joining v to the cloud C . (In Dijkstra’s algorithm, this label measured the full path length from starting vertex s to v , including an edge (u, v) .) These labels serve as keys in a priority queue used to decide which vertex is next in line to join the cloud. We give the pseudo-code in Code Fragment 14.15.

Algorithm PrimJarnik(G):

Input: An undirected, weighted, connected graph G with n vertices and m edges

Output: A minimum spanning tree T for G

Pick any vertex s of G

$D[s] = 0$

for each vertex $v \neq s$ **do**

$D[v] = \infty$

Initialize $T = \emptyset$.

Initialize a priority queue Q with an entry $(D[v], (v, \text{None}))$ for each vertex v , where $D[v]$ is the key in the priority queue, and (v, None) is the associated value.

while Q is not empty **do**

$(u, e) = \text{value returned by } Q.\text{remove_min}()$

 Connect vertex u to T using edge e .

for each edge $e' = (u, v)$ such that v is in Q **do**

 {check if edge (u, v) better connects v to T }

if $w(u, v) < D[v]$ **then**

$D[v] = w(u, v)$

 Change the key of vertex v in Q to $D[v]$.

 Change the value of vertex v in Q to (v, e') .

return the tree T

Code Fragment 14.15: The Prim-Jarník algorithm for the MST problem.

Analyzing the Prim-Jarník Algorithm

The implementation issues for the Prim-Jarník algorithm are similar to those for Dijkstra's algorithm, relying on an adaptable priority queue Q (Section 9.5.1). We initially perform n insertions into Q , later perform n extract-min operations, and may update a total of m priorities as part of the algorithm. Those steps are the primary contributions to the overall running time. With a heap-based priority queue, each operation runs in $O(\log n)$ time, and the overall time for the algorithm is $O((n + m) \log n)$, which is $O(m \log n)$ for a connected graph. Alternatively, we can achieve $O(n^2)$ running time by using an unsorted list as a priority queue.

Illustrating the Prim-Jarník Algorithm

We illustrate the Prim-Jarník algorithm in Figures 14.20 through 14.21.

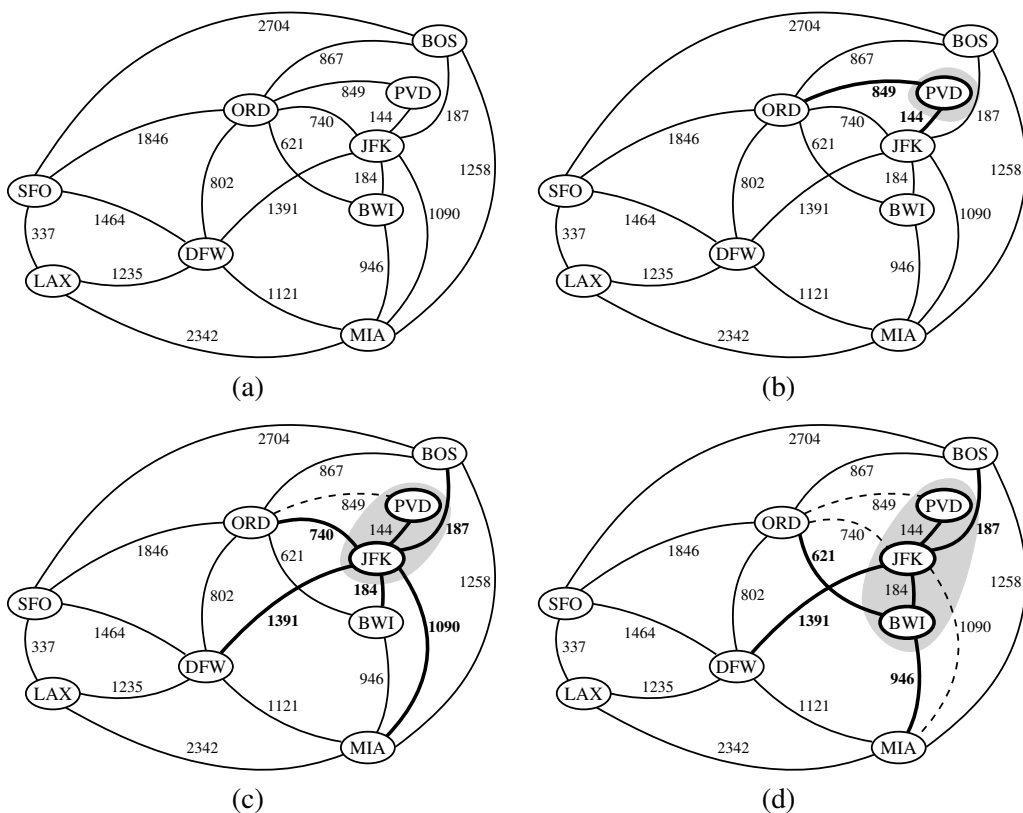


Figure 14.20: An illustration of the Prim-Jarník MST algorithm, starting with vertex PVD. (Continues in Figure 14.21.)

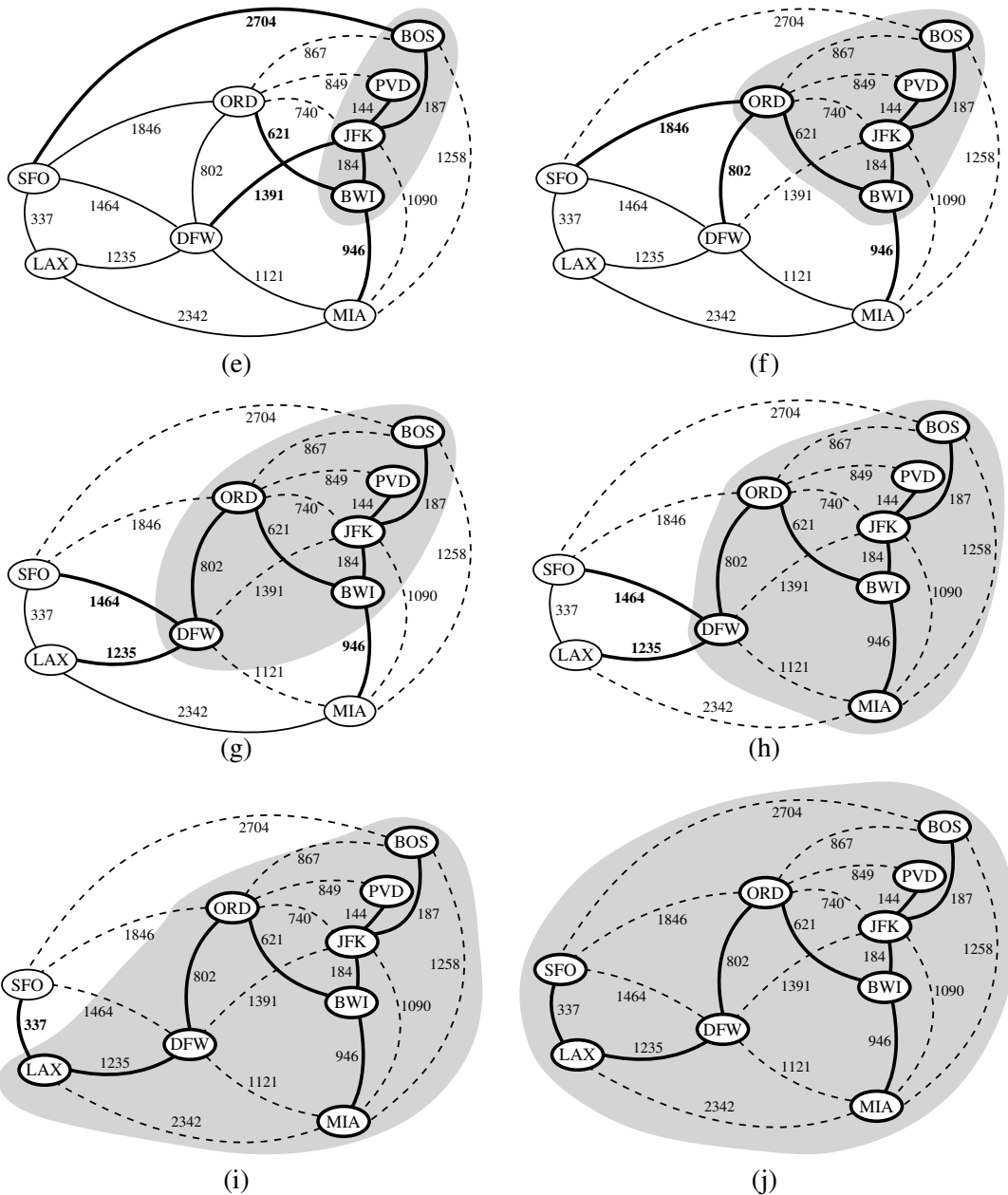


Figure 14.21: An illustration of the Prim-Jarník MST algorithm. (Continued from Figure 14.20.)

Python Implementation

Code Fragment 14.16 presents a Python implementation of the Prim-Jarník algorithm. The MST is returned as an unordered list of edges.

```

1  def MST_PrimJarnik(g):
2      """ Compute a minimum spanning tree of weighted graph g.
3
4      Return a list of edges that comprise the MST (in arbitrary order).
5      """
6      d = { }                                # d[v] is bound on distance to tree
7      tree = [ ]                             # list of edges in spanning tree
8      pq = AdaptableHeapPriorityQueue( )     # d[v] maps to value (v, e=(u,v))
9      pqlocator = { }                       # map from vertex to its pq locator
10
11     # for each vertex v of the graph, add an entry to the priority queue, with
12     # the source having distance 0 and all others having infinite distance
13     for v in g.vertices():
14         if len(d) == 0:                     # this is the first node
15             d[v] = 0                        # make it the root
16         else:
17             d[v] = float('inf')             # positive infinity
18             pqlocator[v] = pq.add(d[v], (v, None))
19
20     while not pq.is_empty():
21         key, value = pq.remove_min()
22         u, edge = value                     # unpack tuple from pq
23         del pqlocator[u]                   # u is no longer in pq
24         if edge is not None:
25             tree.append(edge)               # add edge to tree
26         for link in g.incident_edges(u):
27             v = link.opposite(u)
28             if v in pqlocator:              # thus v not yet in tree
29                 # see if edge (u,v) better connects v to the growing tree
30                 wgt = link.element()
31                 if wgt < d[v]:               # better edge to v?
32                     d[v] = wgt              # update the distance
33                     pq.update(pqlocator[v], d[v], (v, link)) # update the pq entry
34     return tree

```

Code Fragment 14.16: Python implementation of the Prim-Jarník algorithm for the minimum spanning tree problem.

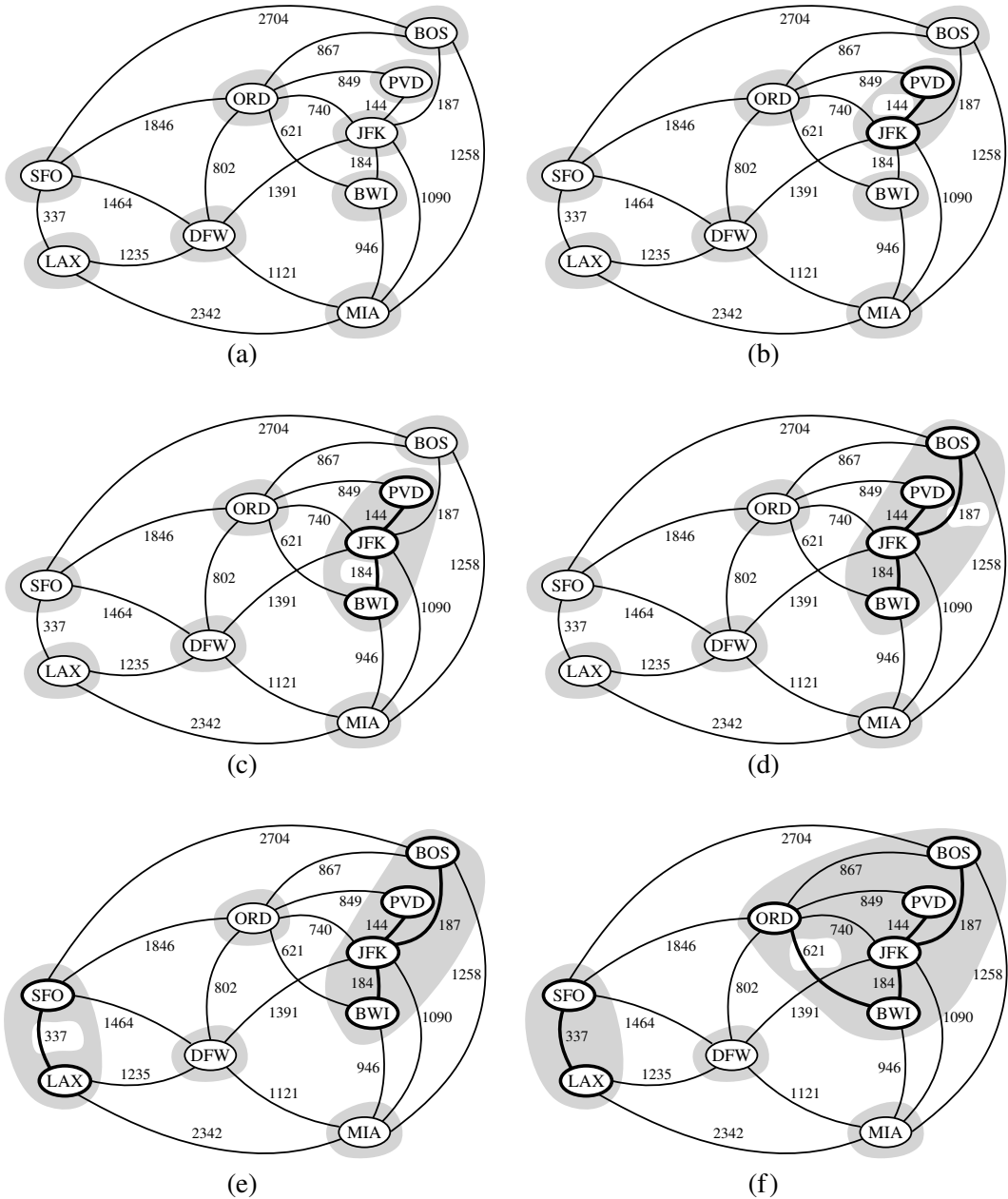


Figure 14.22: Example of an execution of Kruskal's MST algorithm on a graph with integer weights. We show the clusters as shaded regions and we highlight the edge being considered in each iteration. (Continues in Figure 14.23.)

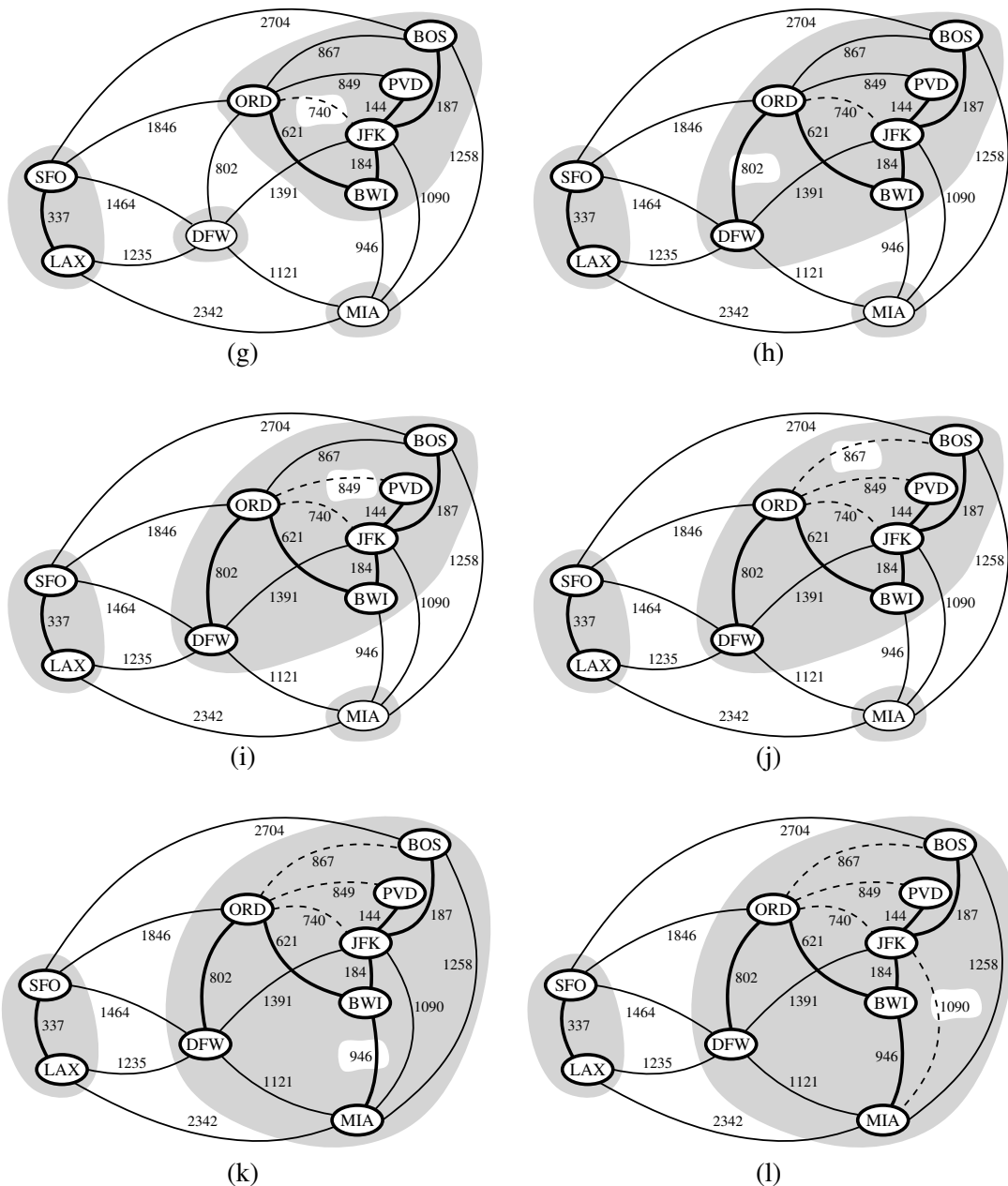


Figure 14.23: An example of an execution of Kruskal's MST algorithm. Rejected edges are shown dashed. (Continues in Figure 14.24.)

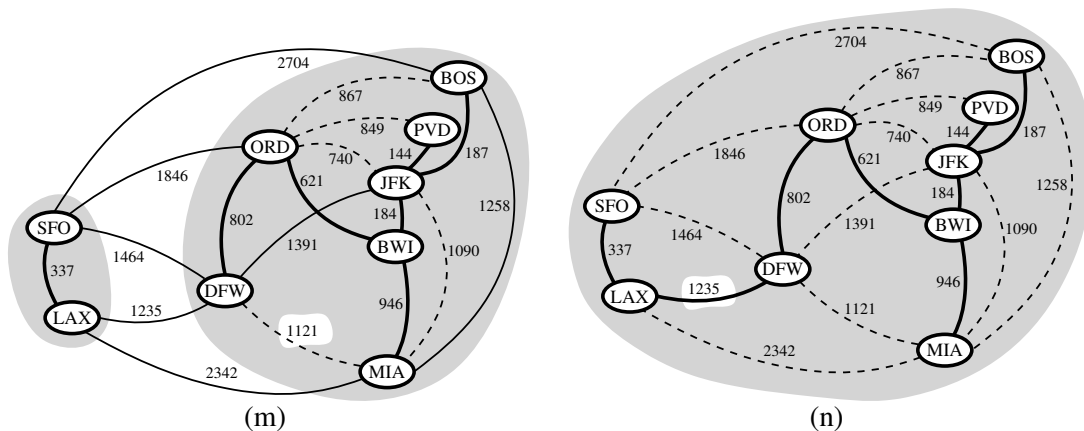


Figure 14.24: Example of an execution of Kruskal's MST algorithm (continued). The edge considered in (n) merges the last two clusters, which concludes this execution of Kruskal's algorithm. (Continued from Figure 14.23.)

The Running Time of Kruskal's Algorithm

There are two primary contributions to the running time of Kruskal's algorithm. The first is the need to consider the edges in nondecreasing order of their weights, and the second is the management of the cluster partition. Analyzing its running time requires that we give more details on its implementation.

The ordering of edges by weight can be implemented in $O(m \log m)$, either by use of a sorting algorithm or a priority queue Q . If that queue is implemented with a heap, we can initialize Q in $O(m \log m)$ time by repeated insertions, or in $O(m)$ time using bottom-up heap construction (see Section 9.3.6), and the subsequent calls to `remove_min` each run in $O(\log m)$ time, since the queue has size $O(m)$. We note that since m is $O(n^2)$ for a simple graph, $O(\log m)$ is the same as $O(\log n)$. Therefore, the running time due to the ordering of edges is $O(m \log n)$.

The remaining task is the management of clusters. To implement Kruskal's algorithm, we must be able to find the clusters for vertices u and v that are endpoints of an edge e , to test whether those two clusters are distinct, and if so, to merge those two clusters into one. None of the data structures we have studied thus far are well suited for this task. However, we conclude this chapter by formalizing the problem of managing *disjoint partitions*, and introducing efficient *union-find* data structures. In the context of Kruskal's algorithm, we perform at most $2m$ find operations and $n - 1$ union operations. We will see that a simple union-find structure can perform that combination of operations in $O(m + n \log n)$ time (see Proposition 14.26), and a more advanced structure can support an even faster time.

For a connected graph, $m \geq n - 1$, and therefore, the bound of $O(m \log n)$ time for ordering the edges dominates the time for managing the clusters. We conclude that the running time of Kruskal's algorithm is $O(m \log n)$.

Python Implementation

Code Fragment 14.18 presents a Python implementation of Kruskal's algorithm. As with our implementation of the Prim-Jarník algorithm, the minimum spanning tree is returned in the form of a list of edges. As a consequence of Kruskal's algorithm, those edges will be reported in nondecreasing order of their weights.

Our implementation assumes use of a `Partition` class for managing the cluster partition. An implementation of the `Partition` class is presented in Section 14.7.3.

```

1  def MST_Kruskal(g):
2      """ Compute a minimum spanning tree of a graph using Kruskal's algorithm.
3
4      Return a list of edges that comprise the MST.
5
6      The elements of the graph's edges are assumed to be weights.
7      """
8      tree = [ ]                # list of edges in spanning tree
9      pq = HeapPriorityQueue( ) # entries are edges in G, with weights as key
10     forest = Partition( )     # keeps track of forest clusters
11     position = { }           # map each node to its Partition entry
12
13     for v in g.vertices():
14         position[v] = forest.make_group(v)
15
16     for e in g.edges():
17         pq.add(e.element(), e) # edge's element is assumed to be its weight
18
19     size = g.vertex_count()
20     while len(tree) != size - 1 and not pq.is_empty():
21         # tree not spanning and unprocessed edges remain
22         weight, edge = pq.remove_min()
23         u, v = edge.endpoints()
24         a = forest.find(position[u])
25         b = forest.find(position[v])
26         if a != b:
27             tree.append(edge)
28             forest.union(a, b)
29
30     return tree

```

Code Fragment 14.18: Python implementation of Kruskal's algorithm for the minimum spanning tree problem.

14.7.3 Disjoint Partitions and Union-Find Structures

In this section, we consider a data structure for managing a *partition* of elements into a collection of disjoint sets. Our initial motivation is in support of Kruskal's minimum spanning tree algorithm, in which a forest of disjoint trees is maintained, with occasional merging of neighboring trees. More generally, the disjoint partition problem can be applied to various models of discrete growth.

We formalize the problem with the following model. A partition data structure manages a universe of elements that are organized into disjoint sets (that is, an element belongs to one and only one of these sets). Unlike with the Set ADT or Python's set class, we do not expect to be able to iterate through the contents of a set, nor to efficiently test whether a given set includes a given element. To avoid confusion with such notions of a set, we will refer to the clusters of our partition as *groups*. However, we will not require an explicit structure for each group, instead allowing the organization of groups to be implicit. To differentiate between one group and another, we assume that at any point in time, each group has a designated entry that we refer to as the *leader* of the group.

Formally, we define the methods of a *partition ADT* using position objects, each of which stores an element x . The partition ADT supports the following methods.

make_group(x): Create a singleton group containing new element x and return the position storing x .

union(p , q): Merge the groups containing positions p and q .

find(p): Return the position of the leader of the group containing position p .

Sequence Implementation

A simple implementation of a partition with a total of n elements uses a collection of sequences, one for each group, where the sequence for a group A stores element positions. Each position object stores a variable, *element*, which references its associated element x and allows the execution of an *element()* method in $O(1)$ time. In addition, each position stores a variable, *group*, that references the sequence storing p , since this sequence is representing the group containing p 's element. (See Figure 14.25.)

With this representation, we can easily perform the *make_group(x)* and *find(p)* operations in $O(1)$ time, allowing the first position in a sequence to serve as the "leader." Operation *union(p , q)* requires that we join two sequences into one and update the group references of the positions in one of the two. We choose to implement this operation by removing all the positions from the sequence with smaller

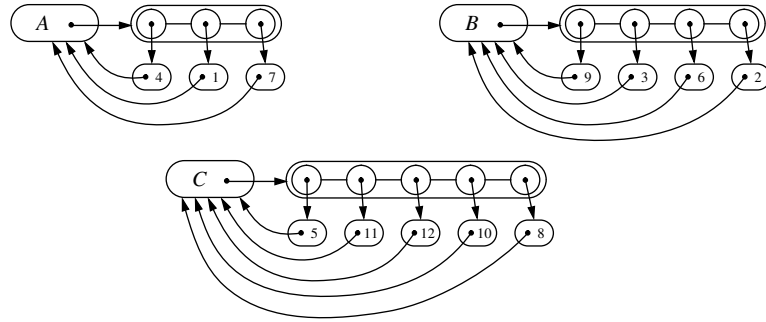


Figure 14.25: Sequence-based implementation of a partition consisting of three groups: $A = \{1, 4, 7\}$, $B = \{2, 3, 6, 9\}$, and $C = \{5, 8, 10, 11, 12\}$.

size, and inserting them in the sequence with larger size. Each time we take a position from the smaller group a and insert it into the larger group b , we update the group reference for that position to now point to b . Hence, the operation $\text{union}(p, q)$ takes time $O(\min(n_p, n_q))$, where n_p (resp. n_q) is the cardinality of the group containing position p (resp. q). Clearly, this time is $O(n)$ if there are n elements in the partition universe. However, we next present an amortized analysis that shows this implementation to be much better than appears from this worst-case analysis.

Proposition 14.26: *When using the above sequence-based partition implementation, performing a series of k `make_group`, `union`, and `find` operations on an initially empty partition involving at most n elements takes $O(k + n \log n)$ time.*

Justification: We use the accounting method and assume that one cyber-dollar can pay for the time to perform a `find` operation, a `make_group` operation, or the movement of a position object from one sequence to another in a `union` operation. In the case of a `find` or `make_group` operation, we charge the operation itself 1 cyber-dollar. In the case of a `union` operation, we assume that 1 cyber-dollar pays for the constant-time work in comparing the sizes of the two sequences, and that we charge 1 cyber-dollar to each position that we move from the smaller group to the larger group. Clearly, the 1 cyber-dollar charged for each `find` and `make_group` operation, together with the first cyber-dollar collected for each `union` operation, accounts for a total of k cyber-dollars.

Consider, then, the number of charges made to positions on behalf of union operations. The important observation is that each time we move a position from one group to another, the size of that position's group at least doubles. Thus, each position is moved from one group to another at most $\log n$ times; hence, each position can be charged at most $O(\log n)$ times. Since we assume that the partition is initially empty, there are $O(n)$ different elements referenced in the given series of operations, which implies that the total time for moving elements during the union operations is $O(n \log n)$. ■

A Tree-Based Partition Implementation ★

An alternative data structure for representing a partition uses a collection of trees to store the n elements, where each tree is associated with a different group. (See Figure 14.26.) In particular, we implement each tree with a linked data structure whose nodes are themselves the group position objects. We view each position p as being a node having an instance variable, element, referring to its element x , and an instance variable, parent, referring to its parent node. By convention, if p is the **root** of its tree, we set p 's parent reference to itself.

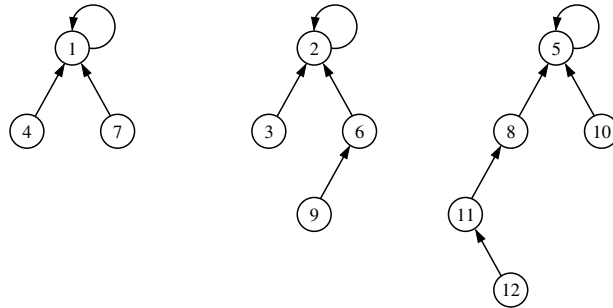


Figure 14.26: Tree-based implementation of a partition consisting of three groups: $A = \{1, 4, 7\}$, $B = \{2, 3, 6, 9\}$, and $C = \{5, 8, 10, 11, 12\}$.

With this partition data structure, operation $\text{find}(p)$ is performed by walking up from position p to the root of its tree, which takes $O(n)$ time in the worst case. Operation $\text{union}(p, q)$ can be implemented by making one of the trees a subtree of the other. This can be done by first locating the two roots, and then in $O(1)$ additional time by setting the parent reference of one root to point to the other root. See Figure 14.27 for an example of both operations.

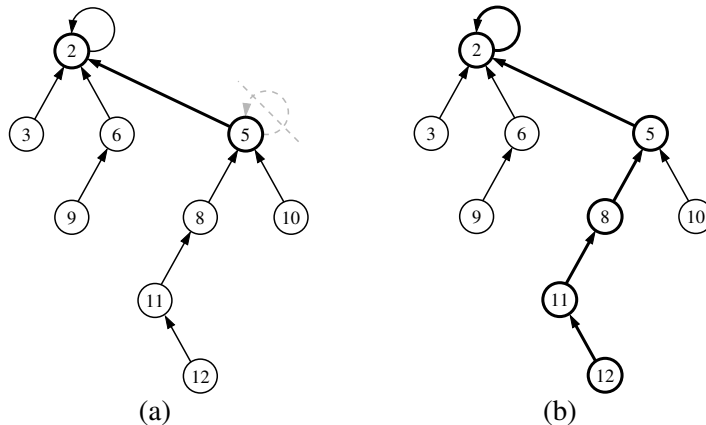


Figure 14.27: Tree-based implementation of a partition: (a) operation $\text{union}(p, q)$; (b) operation $\text{find}(p)$, where p denotes the position object for element 12.

At first, this implementation may seem to be no better than the sequence-based data structure, but we add the following two simple heuristics to make it run faster.

Union-by-Size: With each position p , store the number of elements in the subtree rooted at p . In a union operation, make the root of the smaller group become a child of the other root, and update the size field of the larger root.

Path Compression: In a find operation, for each position q that the find visits, reset the parent of q to the root. (See Figure 14.28.)

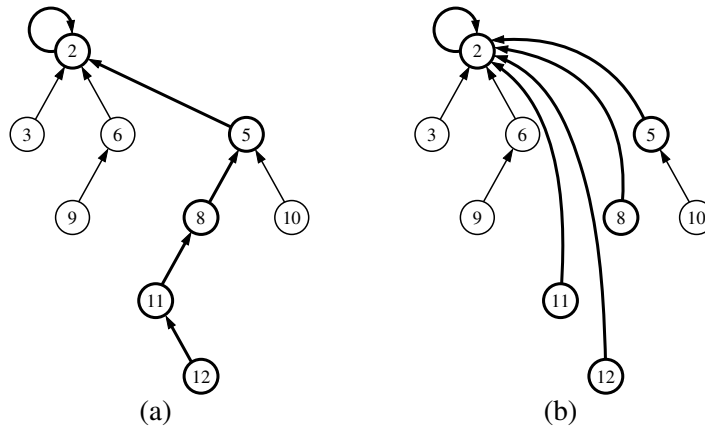


Figure 14.28: Path-compression heuristic: (a) path traversed by operation find on element 12; (b) restructured tree.

A surprising property of this data structure, when implemented using the union-by-size and path-compression heuristics, is that performing a series of k operations involving n elements takes $O(k \log^* n)$ time, where $\log^* n$ is the **log-star** function, which is the inverse of the **tower-of-twos** function. Intuitively, $\log^* n$ is the number of times that one can iteratively take the logarithm (base 2) of a number before getting a number smaller than 2. Table 14.4 shows a few sample values.

minimum n	2	$2^2 = 4$	$2^{2^2} = 16$	$2^{2^{2^2}} = 65,536$	$2^{2^{2^{2^2}}} = 2^{65,536}$
$\log^* n$	1	2	3	4	5

Table 14.4: Some values of $\log^* n$ and critical values for its inverse.

Proposition 14.27: When using the tree-based partition representation with both union-by-size and path compression, performing a series of k make_group, union, and find operations on an initially empty partition involving at most n elements takes $O(k \log^* n)$ time.

Although the analysis for this data structure is rather complex, its implementation is quite straightforward. We conclude with complete Python code for the structure, given in Code Fragment 14.19.

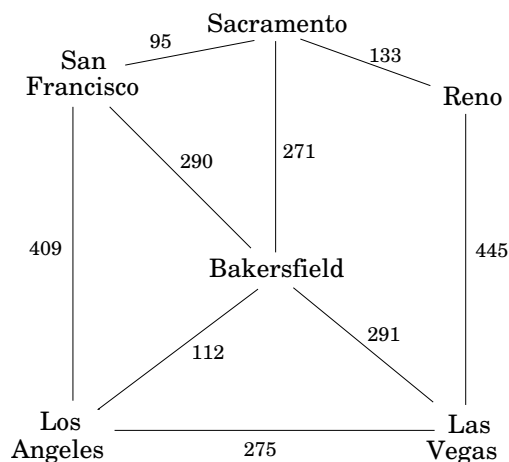
```

1  class Partition:
2      """ Union-find structure for maintaining disjoint sets."""
3
4      #----- nested Position class -----
5      class Position:
6          __slots__ = '_container', '_element', '_size', '_parent'
7
8          def __init__(self, container, e):
9              """ Create a new position that is the leader of its own group."""
10             self._container = container      # reference to Partition instance
11             self._element = e
12             self._size = 1
13             self._parent = self              # convention for a group leader
14
15         def element(self):
16             """ Return element stored at this position."""
17             return self._element
18
19     #----- public Partition methods -----
20     def make_group(self, e):
21         """ Makes a new group containing element e, and returns its Position."""
22         return self.Position(self, e)
23
24     def find(self, p):
25         """ Finds the group containing p and return the position of its leader."""
26         if p._parent != p:
27             p._parent = self.find(p._parent)  # overwrite p._parent after recursion
28         return p._parent
29
30     def union(self, p, q):
31         """ Merges the groups containg elements p and q (if distinct)."""
32         a = self.find(p)
33         b = self.find(q)
34         if a is not b:                        # only merge if different groups
35             if a._size > b._size:
36                 b._parent = a
37                 a._size += b._size
38             else:
39                 a._parent = b
40                 b._size += a._size

```

Code Fragment 14.19: Python implementation of a Partition class using union-by-size and path compression.

Figure 4.5 Edge lengths often matter.



Also notice one stylistic difference from DFS: since we are only interested in distances from s , we do not restart the search in other connected components. Nodes not reachable from s are simply ignored.

4.3 Lengths on edges

Breadth-first search treats all edges as having the same length. This is rarely true in applications where shortest paths are to be found. For instance, suppose you are driving from San Francisco to Las Vegas, and want to find the quickest route. Figure 4.5 shows the major highways you might conceivably use. Picking the right combination of them is a shortest-path problem in which the length of each edge (each stretch of highway) is important. For the remainder of this chapter, we will deal with this more general scenario, annotating every edge $e \in E$ with a length l_e . If $e = (u, v)$, we will sometimes also write $l(u, v)$ or l_{uv} .

These l_e 's do not have to correspond to physical lengths. They could denote time (driving time between cities) or money (cost of taking a bus), or any other quantity that we would like to conserve. In fact, there are cases in which we need to use negative lengths, but we will briefly overlook this particular complication.

4.4 Dijkstra's algorithm

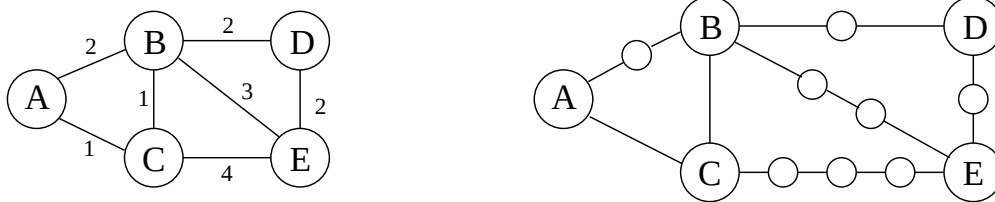
4.4.1 An adaptation of breadth-first search

Breadth-first search finds shortest paths in any graph whose edges have unit length. Can we adapt it to a more general graph $G = (V, E)$ whose edge lengths l_e are *positive integers*?

A more convenient graph

Here is a simple trick for converting G into something BFS can handle: break G 's long edges into unit-length pieces, by introducing “dummy” nodes. Figure 4.6 shows an example of this

Figure 4.6 Breaking edges into unit-length pieces.



transformation. To construct the new graph G' ,

For any edge $e = (u, v)$ of E , replace it by l_e edges of length 1, by adding $l_e - 1$ dummy nodes between u and v .

Graph G' contains all the vertices V that interest us, and the distances between them are exactly the same as in G . Most importantly, the edges of G' all have unit length. Therefore, we can compute distances in G by running BFS on G' .

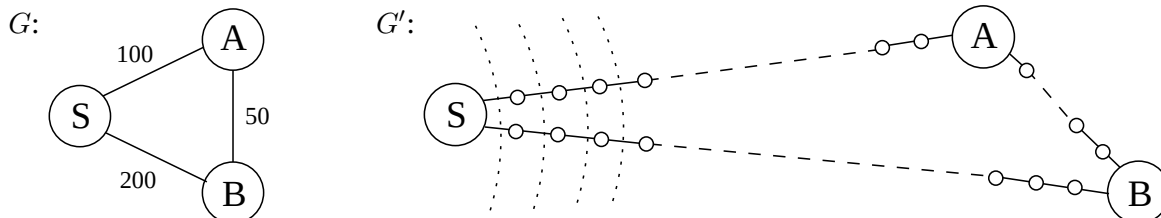
Alarm clocks

If efficiency were not an issue, we could stop here. But when G has very long edges, the G' it engenders is thickly populated with dummy nodes, and the BFS spends most of its time diligently computing distances to these nodes that we don't care about at all.

To see this more concretely, consider the graphs G and G' of Figure 4.7, and imagine that the BFS, started at node s of G' , advances by one unit of distance per minute. For the first 99 minutes it tediously progresses along $S - A$ and $S - B$, an endless desert of dummy nodes. Is there some way we can snooze through these boring phases and have an alarm wake us up whenever something *interesting* is happening—specifically, whenever one of the real nodes (from the original graph G) is reached?

We do this by setting two alarms at the outset, one for node A , set to go off at time $T = 100$, and one for B , at time $T = 200$. These are *estimated times of arrival*, based upon the edges currently being traversed. We doze off and awake at $T = 100$ to find A has been discovered. At this point, the estimated time of arrival for B is adjusted to $T = 150$ and we change its alarm accordingly.

Figure 4.7 BFS on G' is mostly uneventful. The dotted lines show some early “wavefronts.”



More generally, at any given moment the breadth-first search is advancing along certain edges of G , and there is an alarm for every endpoint node toward which it is moving, set to go off at the estimated time of arrival at that node. Some of these might be overestimates because BFS may later find shortcuts, as a result of future arrivals elsewhere. In the preceding example, a quicker route to B was revealed upon arrival at A . However, *nothing interesting can possibly happen before an alarm goes off*. The sounding of the next alarm must therefore signal the arrival of the wavefront to a real node $u \in V$ by BFS. At that point, BFS might also start advancing along some new edges out of u , and alarms need to be set for their endpoints.

The following “alarm clock algorithm” faithfully simulates the execution of BFS on G' .

- Set an alarm clock for node s at time 0.

- Repeat until there are no more alarms:

Say the next alarm goes off at time T , for node u . Then:

- The distance from s to u is T .
- For each neighbor v of u in G :
 - * If there is no alarm yet for v , set one for time $T + l(u, v)$.
 - * If v 's alarm is set for later than $T + l(u, v)$, then reset it to this earlier time.

Dijkstra’s algorithm. The alarm clock algorithm computes distances in any graph with positive integral edge lengths. It is almost ready for use, except that we need to somehow implement the system of alarms. The right data structure for this job is a *priority queue* (usually implemented via a *heap*), which maintains a set of elements (nodes) with associated numeric key values (alarm times) and supports the following operations:

Insert. Add a new element to the set.

Decrease-key. Accommodate the decrease in key value of a particular element.¹

Delete-min. Return the element with the smallest key, and remove it from the set.

Make-queue. Build a priority queue out of the given elements, with the given key values. (In many implementations, this is significantly faster than inserting the elements one by one.)

The first two let us set alarms, and the third tells us which alarm is next to go off. Putting this all together, we get Dijkstra’s algorithm (Figure 4.8).

In the code, $\text{dist}(u)$ refers to the current alarm clock setting for node u . A value of ∞ means the alarm hasn’t so far been set. There is also a special array, `prev`, that holds one crucial piece of information for each node u : the identity of the node immediately before it on the shortest path from s to u . By following these back-pointers, we can easily reconstruct shortest paths, and so this array is a compact summary of all the paths found. A full example of the algorithm’s operation, along with the final shortest-path tree, is shown in Figure 4.9.

In summary, we can think of Dijkstra’s algorithm as just BFS, except it uses a priority queue instead of a regular queue, so as to prioritize nodes in a way that takes edge lengths

¹The name *decrease-key* is standard but is a little misleading: the priority queue typically does not itself change key values. What this procedure really does is to notify the queue that a certain key value has been decreased.

into account. This viewpoint gives a concrete appreciation of how and why the algorithm works, but there is a more direct, more abstract derivation that doesn't depend upon BFS at all. We now start from scratch with this complementary interpretation.

Figure 4.8 Dijkstra's shortest-path algorithm.

procedure `dijkstra`(G, l, s)

Input: Graph $G = (V, E)$, directed or undirected;
 positive edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u reachable from s , `dist`(u) is set
 to the distance from s to u .

for all $u \in V$:
 `dist`(u) = ∞
 `prev`(u) = nil
`dist`(s) = 0

$H = \text{makequeue}(V)$ (using `dist`-values as keys)

while H is not empty:

$u = \text{deletemin}(H)$

for all edges $(u, v) \in E$:

if `dist`(v) > `dist`(u) + $l(u, v)$:
 `dist`(v) = `dist`(u) + $l(u, v)$
 `prev`(v) = u
 `decreasekey`(H, v)

Figure 4.9 A complete run of Dijkstra’s algorithm, with node *A* as the starting point. Also shown are the associated `dist` values and the final shortest-path tree.

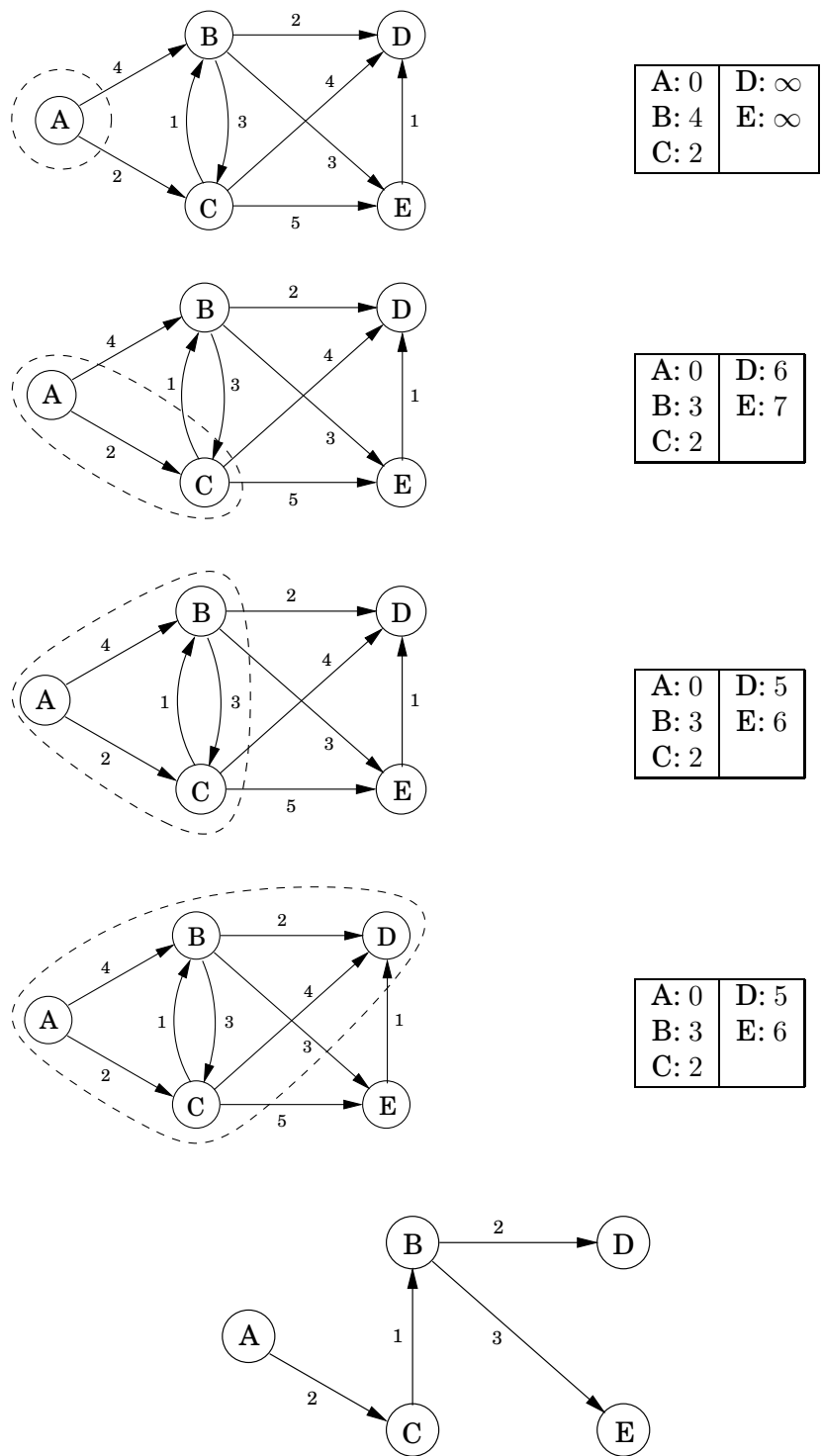
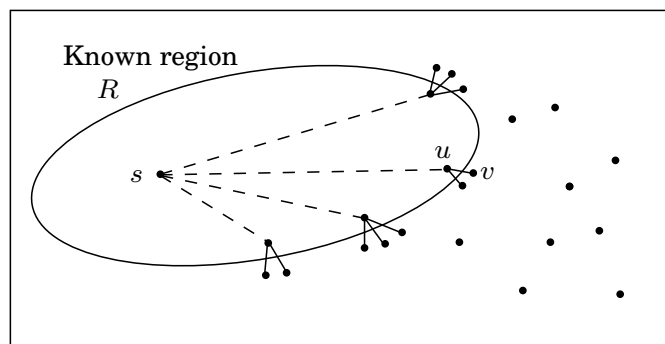


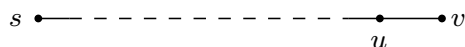
Figure 4.10 Single-edge extensions of known shortest paths.



4.4.2 An alternative derivation

Here's a plan for computing shortest paths: expand outward from the starting point s , steadily growing the region of the graph to which distances and shortest paths are known. This growth should be orderly, first incorporating the closest nodes and then moving on to those further away. More precisely, when the “known region” is some subset of vertices R that includes s , the next addition to it should be *the node outside R that is closest to s* . Let us call this node v ; the question is: how do we identify it?

To answer, consider u , the node just before v in the shortest path from s to v :



Since we are assuming that all edge lengths are positive, u must be closer to s than v is. This means that u is in R —otherwise it would contradict v 's status as the closest node to s outside R . So, the shortest path from s to v is simply *a known shortest path extended by a single edge*.

But there will typically be many single-edge extensions of the currently known shortest paths (Figure 4.10); which of these identifies v ? The answer is, *the shortest of these extended paths*. Because, if an even shorter single-edge-extended path existed, this would once more contradict v 's status as the node outside R closest to s . So, it's easy to find v : it is the node outside R for which the smallest value of $\text{distance}(s, u) + l(u, v)$ is attained, as u ranges over R . In other words, *try all single-edge extensions of the currently known shortest paths, find the shortest such extended path, and proclaim its endpoint to be the next node of R* .

We now have an algorithm for growing R by looking at extensions of the current set of shortest paths. Some extra efficiency comes from noticing that on any given iteration, the only *new* extensions are those involving the node most recently added to region R . All other extensions will have been assessed previously and do not need to be recomputed. In the following pseudocode, $\text{dist}(v)$ is the length of the currently shortest single-edge-extended path leading to v ; it is ∞ for nodes not adjacent to R .

```

Initialize  $\text{dist}(s)$  to 0, other  $\text{dist}(\cdot)$  values to  $\infty$ 
 $R = \{ \}$  (the “known region”)
while  $R \neq V$ :
    Pick the node  $v \notin R$  with smallest  $\text{dist}(\cdot)$ 
    Add  $v$  to  $R$ 
    for all edges  $(v, z) \in E$ :
        if  $\text{dist}(z) > \text{dist}(v) + l(v, z)$ :
             $\text{dist}(z) = \text{dist}(v) + l(v, z)$ 

```

Incorporating priority queue operations gives us back Dijkstra’s algorithm (Figure 4.8).

To justify this algorithm formally, we would use a proof by induction, as with breadth-first search. Here’s an appropriate inductive hypothesis.

At the end of each iteration of the while loop, the following conditions hold: (1) there is a value d such that all nodes in R are at distance $\leq d$ from s and all nodes outside R are at distance $\geq d$ from s , and (2) for every node u , the value $\text{dist}(u)$ is the length of the shortest path from s to u whose intermediate nodes are constrained to be in R (if no such path exists, the value is ∞).

The base case is straightforward (with $d = 0$), and the details of the inductive step can be filled in from the preceding discussion.

4.4.3 Running time

At the level of abstraction of Figure 4.8, Dijkstra’s algorithm is structurally identical to breadth-first search. However, it is slower because the priority queue primitives are computationally more demanding than the constant-time `eject`’s and `inject`’s of BFS. Since `makequeue` takes at most as long as $|V|$ `insert` operations, we get a total of $|V|$ `deletemin` and $|V| + |E|$ `insert/decreasekey` operations. The time needed for these varies by implementation; for instance, a binary heap gives an overall running time of $O((|V| + |E|) \log |V|)$.

Which heap is best?

The running time of Dijkstra's algorithm depends heavily on the priority queue implementation used. Here are the typical choices.

Implementation	deletemin	insert/ decreasekey	$ V \times \text{deletemin} + (V + E) \times \text{insert}$
Array	$O(V)$	$O(1)$	$O(V ^2)$
Binary heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
d -ary heap	$O(\frac{d \log V }{\log d})$	$O(\frac{\log V }{\log d})$	$O((V \cdot d + E) \frac{\log V }{\log d})$
Fibonacci heap	$O(\log V)$	$O(1)$ (amortized)	$O(V \log V + E)$

So for instance, even a naive array implementation gives a respectable time complexity of $O(|V|^2)$, whereas with a binary heap we get $O((|V| + |E|) \log |V|)$. Which is preferable?

This depends on whether the graph is *sparse* (has few edges) or *dense* (has lots of them). For all graphs, $|E|$ is less than $|V|^2$. If it is $\Omega(|V|^2)$, then clearly the array implementation is the faster. On the other hand, the binary heap becomes preferable as soon as $|E|$ dips below $|V|^2 / \log |V|$.

The d -ary heap is a generalization of the binary heap (which corresponds to $d = 2$) and leads to a running time that is a function of d . The optimal choice is $d \approx |E|/|V|$; in other words, to optimize we must set the degree of the heap to be equal to the *average degree* of the graph. This works well for both sparse and dense graphs. For very sparse graphs, in which $|E| = O(|V|)$, the running time is $O(|V| \log |V|)$, as good as with a binary heap. For dense graphs, $|E| = \Omega(|V|^2)$ and the running time is $O(|V|^2)$, as good as with a linked list. Finally, for graphs with intermediate density $|E| = |V|^{1+\delta}$, the running time is $O(|E|)$, linear!

The last line in the table gives running times using a sophisticated data structure called a *Fibonacci heap*. Although its efficiency is impressive, this data structure requires considerably more work to implement than the others, and this tends to dampen its appeal in practice. We will say little about it except to mention a curious feature of its time bounds. Its insert operations take varying amounts of time but are guaranteed to *average* $O(1)$ over the course of the algorithm. In such situations (one of which we shall encounter in Chapter 5) we say that the *amortized* cost of heap insert's is $O(1)$.

4.5 Priority queue implementations

4.5.1 Array

The simplest implementation of a priority queue is as an unordered array of key values for all potential elements (the vertices of the graph, in the case of Dijkstra's algorithm). Initially, these values are set to ∞ .

An `insert` or `decreasekey` is fast, because it just involves adjusting a key value, an $O(1)$ operation. To `deletemin`, on the other hand, requires a linear-time scan of the list.

4.5.2 Binary heap

Here elements are stored in a *complete* binary tree, namely, a binary tree in which each level is filled in from left to right, and must be full before the next level is started. In addition, a special ordering constraint is enforced: *the key value of any node of the tree is less than or equal to that of its children*. In particular, therefore, the root always contains the smallest element. See Figure 4.11(a) for an example.

To `insert`, place the new element at the bottom of the tree (in the first available position), and let it “bubble up.” That is, if it is smaller than its parent, swap the two and repeat (Figure 4.11(b)–(d)). The number of swaps is at most the height of the tree, which is $\lfloor \log_2 n \rfloor$ when there are n elements. A `decreasekey` is similar, except that the element is already in the tree, so we let it bubble up from its current position.

To `deletemin`, return the root value. To then remove this element from the heap, take the last node in the tree (in the rightmost position in the bottom row) and place it at the root. Let it “sift down”: if it is bigger than either child, swap it with the smaller child and repeat (Figure 4.11(e)–(g)). Again this takes $O(\log n)$ time.

The regularity of a complete binary tree makes it easy to represent using an array. The tree nodes have a natural ordering: row by row, starting at the root and moving left to right within each row. If there are n nodes, this ordering specifies their positions $1, 2, \dots, n$ within the array. Moving up and down the tree is easily simulated on the array, using the fact that node number j has parent $\lfloor j/2 \rfloor$ and children $2j$ and $2j + 1$ (Exercise 4.16).

4.5.3 d -ary heap

A d -ary heap is identical to a binary heap, except that nodes have d children instead of just two. This reduces the height of a tree with n elements to $\Theta(\log_d n) = \Theta((\log n)/(\log d))$. Inserts are therefore speeded up by a factor of $\Theta(\log d)$. `Deletemin` operations, however, take a little longer, namely $O(d \log_d n)$ (do you see why?).

The array representation of a binary heap is easily extended to the d -ary case. This time, node number j has parent $\lceil (j - 1)/d \rceil$ and children $\{(j - 1)d + 2, \dots, \min\{n, (j - 1)d + d + 1\}\}$ (Exercise 4.16).

Figure 4.11 (a) A binary heap with 10 elements. Only the key values are shown. (b)–(d) The intermediate “bubble-up” steps in inserting an element with key 7. (e)–(g) The “sift-down” steps in a delete-min operation.

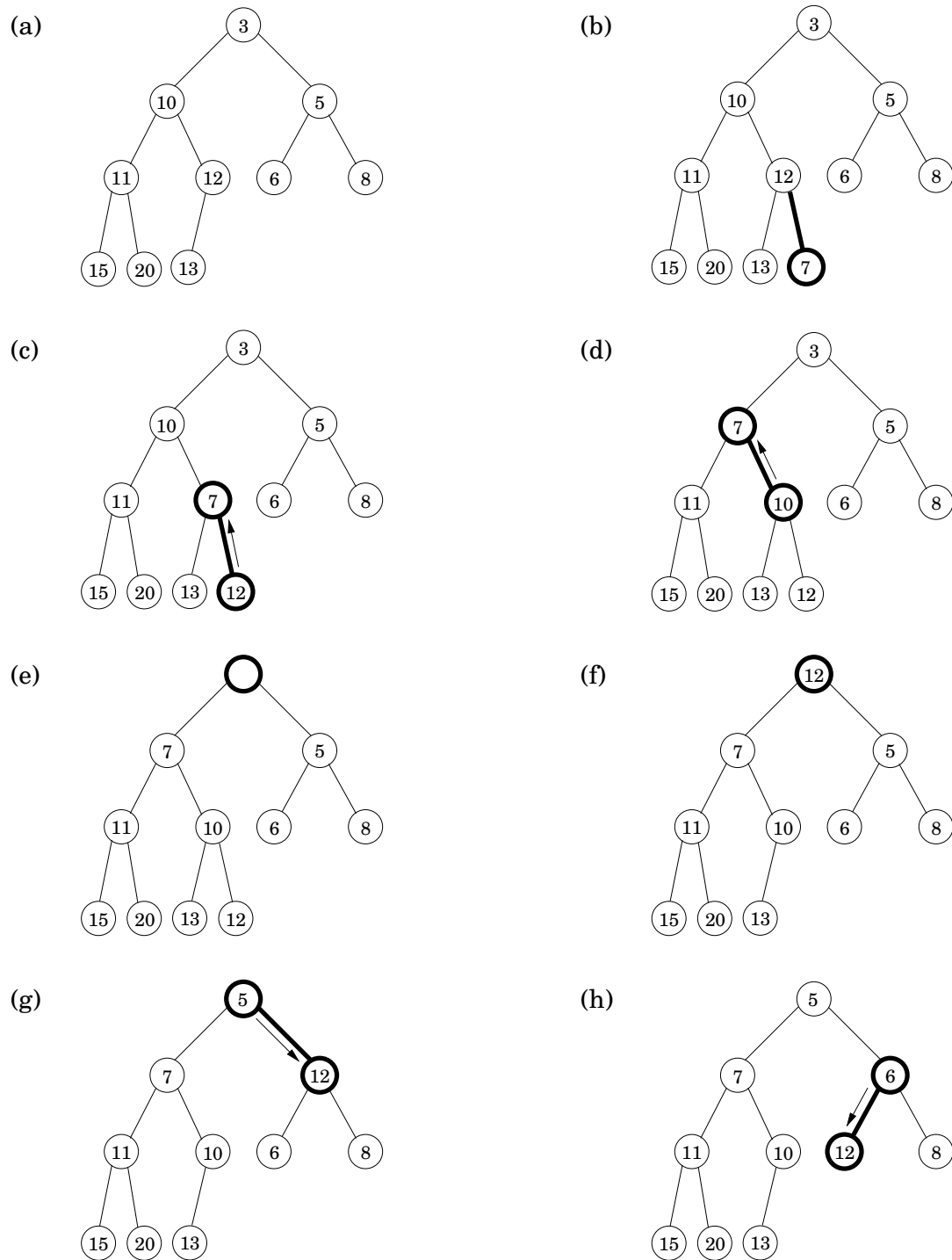
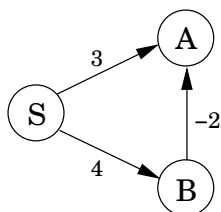


Figure 4.12 Dijkstra’s algorithm will not work if there are negative edges.



4.6 Shortest paths in the presence of negative edges

4.6.1 Negative edges

Dijkstra’s algorithm works in part because the shortest path from the starting point s to any node v must pass exclusively through nodes that are closer than v . This no longer holds when edge lengths can be negative. In Figure 4.12, the shortest path from S to A passes through B , a node that is further away!

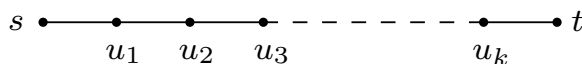
What needs to be changed in order to accommodate this new complication? To answer this, let’s take a particular high-level view of Dijkstra’s algorithm. A crucial invariant is that the dist values it maintains are always either overestimates or exactly correct. They start off at ∞ , and the only way they ever change is by updating along an edge:

procedure $\text{update}((u, v) \in E)$
 $\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u, v)\}$

This *update* operation is simply an expression of the fact that the distance to v cannot possibly be more than the distance to u , plus $l(u, v)$. It has the following properties.

1. It gives the correct distance to v in the particular case where u is the second-last node in the shortest path to v , and $\text{dist}(u)$ is correctly set.
2. It will never make $\text{dist}(v)$ too small, and in this sense it is *safe*. For instance, a slew of extraneous update’s can’t hurt.

This operation is extremely useful: it is harmless, and if used carefully, will correctly set distances. In fact, Dijkstra’s algorithm can be thought of simply as a sequence of update’s. We know this particular sequence doesn’t work with negative edges, but is there some other sequence that does? To get a sense of the properties this sequence must possess, let’s pick a node t and look at the shortest path to it from s .



This path can have at most $|V| - 1$ edges (do you see why?). If the sequence of updates performed includes $(s, u_1), (u_1, u_2), (u_2, u_3), \dots, (u_k, t)$, *in that order* (though not necessarily consecutively), then by the first property the distance to t will be correctly computed. It doesn’t

Figure 4.13 The Bellman-Ford algorithm for single-source shortest paths in general graphs.

```
procedure shortest-paths( $G, l, s$ )  
Input:   Directed graph  $G = (V, E)$ ;  
         edge lengths  $\{l_e : e \in E\}$  with no negative cycles;  
         vertex  $s \in V$   
Output:  For all vertices  $u$  reachable from  $s$ ,  $\text{dist}(u)$  is set  
         to the distance from  $s$  to  $u$ .  
  
for all  $u \in V$ :  
     $\text{dist}(u) = \infty$   
     $\text{prev}(u) = \text{nil}$   
  
 $\text{dist}(s) = 0$   
repeat  $|V| - 1$  times:  
    for all  $e \in E$ :  
        update( $e$ )
```

matter what other updates occur on these edges, or what happens in the rest of the graph, because updates are *safe*.

But still, if we don't know all the shortest paths beforehand, how can we be sure to update the right edges in the right order? Here is an easy solution: simply update *all* the edges, $|V| - 1$ times! The resulting $O(|V| \cdot |E|)$ procedure is called the Bellman-Ford algorithm and is shown in Figure 4.13, with an example run in Figure 4.14.

A note about implementation: for many graphs, the maximum number of edges in any shortest path is substantially less than $|V| - 1$, with the result that fewer rounds of updates are needed. Therefore, it makes sense to add an extra check to the shortest-path algorithm, to make it terminate immediately after any round in which no update occurred.

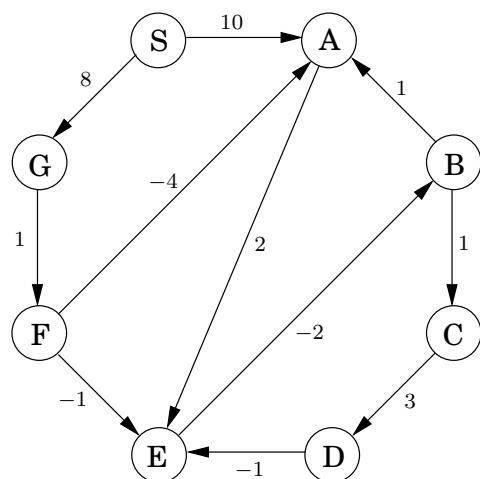
4.6.2 Negative cycles

If the length of edge (E, B) in Figure 4.14 were changed to -4 , the graph would have a *negative cycle* $A \rightarrow E \rightarrow B \rightarrow A$. In such situations, it doesn't make sense to even ask about shortest paths. There is a path of length 2 from A to E . But going round the cycle, there's also a path of length 1, and going round multiple times, we find paths of lengths 0, -1 , -2 , and so on.

The shortest-path problem is ill-posed in graphs with negative cycles. As might be expected, our algorithm from Section 4.6.1 works only in the absence of such cycles. But where did this assumption appear in the derivation of the algorithm? Well, it slipped in when we asserted the *existence* of a shortest path from s to t .

Fortunately, it is easy to automatically detect negative cycles and issue a warning. Such a cycle would allow us to endlessly apply rounds of update operations, reducing dist estimates every time. So instead of stopping after $|V| - 1$ iterations, perform one extra round. There is a negative cycle if and only if some dist value is reduced during this final round.

Figure 4.14 The Bellman-Ford algorithm illustrated on a sample graph.



	Iteration							
Node	0	1	2	3	4	5	6	7
S	0	0	0	0	0	0	0	0
A	∞	10	10	5	5	5	5	5
B	∞	∞	∞	10	6	5	5	5
C	∞	∞	∞	∞	11	7	6	6
D	∞	∞	∞	∞	∞	14	10	9
E	∞	∞	12	8	7	7	7	7
F	∞	∞	9	9	9	9	9	9
G	∞	8	8	8	8	8	8	8

4.7 Shortest paths in dags

There are two subclasses of graphs that automatically exclude the possibility of negative cycles: graphs without negative edges, and graphs without cycles. We already know how to efficiently handle the former. We will now see how the single-source shortest-path problem can be solved in just linear time on directed acyclic graphs.

As before, we need to perform a sequence of updates that includes every shortest path as a subsequence. The key source of efficiency is that

In any path of a dag, the vertices appear in increasing linearized order.

Therefore, it is enough to linearize (that is, topologically sort) the dag by depth-first search, and then visit the vertices in sorted order, updating the edges out of each. The algorithm is given in Figure 4.15.

Notice that our scheme doesn't require edges to be positive. In particular, we can find *longest paths* in a dag by the same algorithm: just negate all edge lengths.

Figure 4.15 A single-source shortest-path algorithm for directed acyclic graphs.

procedure dag-shortest-paths(G, l, s)

Input: Dag $G = (V, E)$;

 edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set
 to the distance from s to u .

for all $u \in V$:

$\text{dist}(u) = \infty$

$\text{prev}(u) = \text{nil}$

$\text{dist}(s) = 0$

Linearize G

for each $u \in V$, in linearized order:

 for all edges $(u, v) \in E$:

 update(u, v)
